

إدارة المشروعات التعاونية باستخدام GitHub

1. المفاهيم الأساسية للبيئة السحابية

- SQL، المجلد السحابي الرئيسي لحفظ كل ملفات المشروع (بيانات، أكواد (Repository) المستودع Python، ملفات Power BI، سكربتات).
- Markdown) (README.md ملف شرح الواجهة التعريفية الأولى للمشروع وتُكتب بلغة التنسيق المباشر للمشكلة، الأسئلة التحليلية، والنتائج.
- لتوزيع المهام على (To Do - In Progress - Done) لوحة مهام مرئية (Projects) إدارة المشاريع الفريق ومتابعة التنفيذ.

2. طبيعة التعامل مع الملفات داخل نظام Git

- يتتبعها. SQL (.sql) واستعلامات Python (.py) مثل ملفات (Plain Text Files) الملفات النصية سطرًا بسطر، مما يسمح لأكثر من عضو بالتعديل في نفس الوقت ودمج أسطرهم آلياً دون فقدان أي جهد Git.
- Excel (.xlsx) وملفات Power BI (.pbix) مثل ملفات (Binary Files) الملفات الثنائية/المغلقة قراءة تفاصيلها الداخلية، وبالتالي يجب أن يعمل عليها شخص واحد فقط في كل مرة لتجنب إلغاء Git لا يستطيع تعديلات الآخرين.

3. وقواعد الحماية (Branching Strategy) نظام الفروع

- يمثل النسخة المعتمدة النهائية والخالية من الأخطاء للمشروع (main) الفرع الرئيسي.
- بيانات عمل معزولة ينشئها كل عضو لتجربة وتنفيذ مهمته دون المساس (Branches) الفروع الجانبية بالنسخة الأصلية.
- (Branch Rulesets) قواعد حماية الفرع الرئيسي:
 1. منع الرفع المباشر: إجبار الجميع على رفع التعديلات عبر الفروع الجانبية فقط.
 2. (Pull Request) اشتراط المراجعة: main عدم دمج أي فرع في.
 3. تعيين مدير المشروع كصاحب الموافقة الحصرية والنهائية على (CODEOWNERS) تحديد مالك الكود main. دخول أي ملف أو أسطر جديدة إلى.
 4. لمنع مسح الفرع الرئيسي بالخطأ Restrict deletions منع الحذف: تفعيل شرط.

والعمل الميداني خطوة بخطوة Git القسم الثالث: دليل أوامر

1. (**gitignore**) إعداد البيئة المحلية وتجاهل الملفات المؤقتة

داخل المجلد الرئيسي لتجاهل رفع الملفات المؤقتة أو الحساسة، مثل **gitignore**. يتم إنشاء ملف باسم

كلمات السر والبيانات الحساسة #

.env

ملفات النظام المؤقتة

Thumbs.db

.DS_Store

ملفات بايثون المؤقتة

__pycache__/

.ipynb_checkpoints/

(ملفات البيانات الضخمة) اختياري

*.csv

*.xlsx

2. دورة العمل المباشرة لفريق العمل

GitHub أ. رفع ملف جديد أو تعديل من جهاز الزميل إلى

1. **Git** داخل مجلد المشروع بالضغط على زر الفأرة الأيمن ثم اختيار **Git Bash** فتح الشاشة السوداء **Bash Here**.

2. إنشاء فرع جديد والانتقال إليه:

git checkout -b upload-new-file

إضافة الملفات والتأكيد على الحفظ محلياً:

git add .

3. إضافة الملف الرئيسي للبيانات " git commit -m

4. رفع الفرع إلى موقع **GitHub**:

git push -u origin upload-new-file

5. لإرسال طلب **Compare & pull request** بالمتصفح والضغط على **GitHub** الذهاب إلى موقع **الدمج لمدير المشروع**.

(Gatekeeper) ب. مراجعة واكتساب الملفات بواسطة مدير المشروع

1. **Pull requests.** يراجع مدير المشروع الطلب تحت تبويب.
2. **Approve.** ثم يختار **Review changes** بعد التأكد من سلامة الكود والملفات، يضغط على.
3. لضغط التعديلات الصغيرة في إنجاز **Squash and merge** يفضل استخدام طريقة الدمج بالضغط. واحد نظيف داخل سجل المشروع.

إلى أجهزة باقي الفريق **main** ج. سحب أحدث النسخة المعتمدة من

:لضمان عمل جميع الأعضاء على أحدث البيانات المعتمدة محلياً

1. الانتقال للفرع الرئيسي محلياً:
`git checkout main`
2. سحب جميع التحديثات الجديدة بكلمة واحدة:
`git pull`

3. الميدانية Git ملخص أهم أوامر

الوظيفة	الأمر
استنساخ المستودع لأول مرة على الجهاز المحلي باستخدام العنوان.	<code>git clone <URL></code>
إنشاء فرع جديد والانتقال للعمل داخله فوراً.	<code>git checkout -b <branch-name></code>
تجهيز كافة الملفات المعدلة للنسخ والحفظ.	<code>git add .</code>
حفظ لقطة دائمة للتعديلات محلياً مع كتابة ملاحظة توضيحية.	<code>git commit -m "الوصف"</code>
رفع الفرع والتعديلات إلى السحابة لأول مرة.	<code>git push -u origin <branch-name></code>
الانتقال للعمل على الفرع الرئيسي.	<code>git checkout main</code>
جلب وسحب أحدث الملفات من السحابة إلى الجهاز المحلي.	<code>git pull</code>